

**LAPORAN MACHINE LEARNING
DECISION TREE VS NAIVE BAYES**



Disusun Oleh:

Anjelia Febiliyanti

G1A024053

Kelas:

Informatika (A)

Dosen Pengampu:

Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS BENGKULU
2026**

DESKRIPSI TUGAS

Kamu adalah seorang Data Scientist di startup EduTech. Perusahaan ingin memprediksi "Apakah mahasiswa akan LULUS TEPAT WAKTU atau TIDAK". Dataset: mahasiswa_lulus.csv - 500 data dengan fitur: IPK, Kehadiran, Jam_Belajar, Organisasi, Penghasilan_Ortu, Jenis_Kelamin, Status_Beasiswa

BAGIAN A: ANALISIS KONSEP

1. Jelaskan kapan Decision Tree lebih cocok digunakan dibanding Naive Bayes untuk kasus prediksi kelulusan ini. Beri 2 alasan.

Decision Tree lebih cocok digunakan jika kita ingin mendapatkan hasil prediksi yang mudah dipahami. Dalam kasus prediksi kelulusan mahasiswa, Decision Tree dapat menunjukkan aturan berdasarkan beberapa fitur seperti IPK, kehadiran, dan jam belajar. Misalnya, dari hasil percabangan dapat diketahui bahwa mahasiswa dengan IPK dan kehadiran yang tinggi lebih berpeluang untuk lulus tepat waktu. Aturan seperti ini akan lebih mudah dijelaskan dibandingkan hasil dari model yang perhitungannya lebih sulit dilihat secara langsung.

Selain itu, Decision Tree tidak mengharuskan setiap fitur dianggap saling independen. Hal ini cukup penting karena pada dataset ini kemungkinan terdapat hubungan antarfitur, misalnya IPK dengan jam belajar atau IPK dengan kehadiran. Decision Tree tetap dapat menggunakan hubungan tersebut dalam menentukan percabangan data. Jadi, Decision Tree lebih cocok jika membutuhkan model yang hasilnya mudah dijelaskan dan tidak terlalu bergantung pada asumsi independensi antarfitur.

2. Asumsi "independen" di Naive Bayes. Sebutkan 2 pasang fitur di dataset yang kemungkinan melanggar asumsi ini dan jelaskan kenapa.

Naive Bayes memiliki asumsi bahwa setiap fitur dianggap independen satu sama lain. Namun, pada dataset ini ada beberapa fitur yang kemungkinan saling berhubungan. Contohnya adalah IPK dan Jam_Belajar. Mahasiswa yang memiliki waktu belajar lebih banyak biasanya memiliki kesempatan lebih besar untuk memahami materi perkuliahan, sehingga hal tersebut bisa berpengaruh terhadap IPK. Karena itu, kedua fitur tersebut kemungkinan tidak benar-benar independen.

Pasangan lainnya adalah IPK dan Kehadiran. Kehadiran yang tinggi membuat mahasiswa lebih sering mengikuti perkuliahan dan mendapatkan materi secara langsung dari dosen. Kondisi tersebut dapat berhubungan dengan nilai akademik atau IPK yang diperoleh. Jadi, IPK dan kehadiran juga kemungkinan memiliki

hubungan, sehingga tidak sepenuhnya memenuhi asumsi independensi pada Naive Bayes.

3. Apa risiko overfitting pada Decision Tree di dataset ini? Bagaimana cara mengatasinya?

Overfitting dapat terjadi ketika Decision Tree terlalu mengikuti pola yang ada pada data training. Jika pohonnya dibuat tanpa batas seperti menggunakan `max_depth=None`, pohon dapat menghasilkan terlalu banyak percabangan dan aturan yang sangat spesifik. Akibatnya, model mungkin terlihat bagus saat digunakan pada data training, tetapi ketika diberikan data baru, hasil prediksinya justru kurang baik. Hal ini dapat dilihat dari hasil pengujian pada laporan, yaitu Decision Tree dengan `max_depth=None` hanya mendapatkan akurasi 66%, sedangkan Decision Tree dengan `max_depth=3` mendapatkan akurasi 71%.

Salah satu cara untuk mengurangi overfitting adalah membatasi kedalaman pohon menggunakan parameter `max_depth`. Pada tugas ini digunakan `max_depth=3`, sehingga jumlah percabangan pohon tidak terlalu banyak dan model menjadi lebih sederhana. Selain itu, kita juga dapat menggunakan parameter seperti `min_samples_split` dan `min_samples_leaf` untuk membatasi percabangan yang terlalu detail. Pengujian menggunakan data testing juga penting agar kita dapat melihat apakah model mampu memberikan prediksi yang baik pada data yang belum pernah digunakan saat proses training.

BAGIAN B : IMPLEMENTASI & EVALUASI

Gunakan Python + sklearn + pandas + matplotlib

1. Preprocessing: Handle missing value, encoding, train-test split 80:20

Jawaban:

Source Code :

```
1 # Importing Libraries
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 from sklearn.preprocessing import LabelEncoder
6 from sklearn.tree import DecisionTreeClassifier, plot_tree
7 from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
8 confusion_matrix, ConfusionMatrixDisplay, classification_report
9
10 # Load Data
11 df = pd.read_csv('data/katastrofis_data.csv')
12 print(df.head())
13 print(df.info())
14 print(df.isnull().sum())
15 print(df.describe())
```

```
1 # Preprocessing
2 # 1. Handle missing values
3 # 2. PREPROCESSING
4 # 1. Handle missing values
5 # 2. Encoding categorical variables
6 # 3. Train-Test Split
7 # 4. Model Training
8 # 5. Model Evaluation
```

Output :

```
1. DATA PENTAH
=====
Jumlah baris x kolom : (500, 8)

Info tipe data & missing value:
IPK          20
Kehadiran    20
Jam_Belajar  20
Organisasi   20
Penghasilan_Ortu 0
Jenis_Kelamin 0
Status_Beasiswa 0
Lulus_Tepat_Waktu 0
dtype: int64

Contoh 5 baris pertama:
   IPK  Kehadiran  Jam_Belajar  Organisasi  Penghasilan_Ortu  Jenis_Kelamin \
0  3.32    93.1      6.3      Aktif      Menengah      Perempuan
1  3.04    100.0     5.7  Tidak Aktif      Rendah      Perempuan
2  3.39     65.2     3.1  Tidak Aktif      Menengah      Perempuan
3  3.79     88.8     2.5      Aktif      Rendah      Laki-laki
4  2.99     74.2     4.6      Aktif      Menengah      Laki-laki
```

```
Status_Beasiswa Lulus_Tepat_Waktu
0              Ya      Tepat Waktu
1             Tidak      Tepat Waktu
2             Tidak      Tepat Waktu
3              Ya      Tepat Waktu
4             Tidak      Tepat Waktu

2. PREPROCESSING
=====
Missing 'IPK' diisi dengan median = 3.105
Missing 'Kehadiran' diisi dengan median = 82.3
Missing 'Jam_Belajar' diisi dengan median = 3.7
Missing 'Organisasi' diisi dengan modus = Tidak Aktif

Total missing value setelah dibersihkan: 0
Encoding 'Organisasi': {'Aktif': np.int64(0), 'Tidak Aktif': np.int64(1)}
Encoding 'Penghasilan_Ortu': {'Menengah': np.int64(0), 'Rendah': np.int64(1), 'Tinggi': np.int64(2)}
Encoding 'Jenis_Kelamin': {'Laki-laki': np.int64(0), 'Perempuan': np.int64(1)}
Encoding 'Status_Beasiswa': {'Tidak': np.int64(0), 'Ya': np.int64(1)}
Encoding target 'Lulus_Tepat_Waktu': {'Tepat Waktu': np.int64(0), 'Tidak Tepat Waktu': np.int64(1)}

Jumlah data latih : 400
Jumlah data uji   : 100
```

Penjelasan :

Pemrosesan data dimulai dari penghubung Colab ke Drive, pemuatan dataset 500 baris, penanganan *missing value*, hingga *encoding* dan *train-test split* 80:20. Selanjutnya, tiga model (*Decision Tree* *max_depth=3*, *max_depth=None*, dan *Gaussian Naive Bayes*) dilatih serta dievaluasi menggunakan metrik akurasi, *precision*, *recall*, *F1-score*, dan *confusion matrix*. Terakhir, dibuat visualisasi pohon keputusan serta diagram batang metrik, disertai analisis error pada 5 data uji yang salah diprediksi.

2. Modeling:

Bangun model *DecisionTreeClassifier* dengan *max_depth=3* dan *max_depth=None*
Bangun model *GaussianNB*

Jawabana :

Source Code dan Output :

```
3. MODELING
=====
Model 'Decision Tree (max_depth=3)' selesai dilatih.
Model 'Decision Tree (max_depth=None)' selesai dilatih.
Model 'Gaussian Naive Bayes' selesai dilatih.
```

```
3. MODELING
=====
Model 'Decision Tree (max_depth=3)' selesai dilatih.
Model 'Decision Tree (max_depth=None)' selesai dilatih.
Model 'Gaussian Naive Bayes' selesai dilatih.
```

Penjelasan :

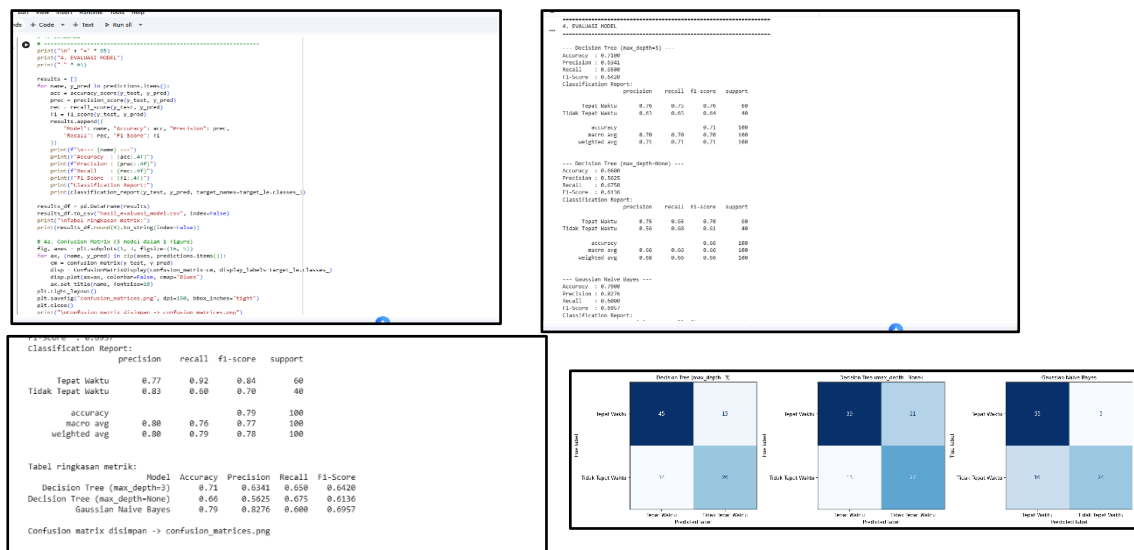
Tahap *modeling* menginisialisasi dan melatih tiga algoritma klasifikasi—*Decision Tree* (*max_depth=3*), *Decision Tree* tanpa batas (*max_depth=None*), dan *Gaussian Naive Bayes*—menggunakan data latih, lalu menghasilkan prediksi pada data uji secara otomatis melalui perulangan (*looping*).

3. Evaluasi: Hitung Akurasi, Precision, Recall, F1-Score untuk ke-3 model.

Buat *Confusion Matrix*

Jawaban :

Source Code Output :



Penjelasan :

Tahap evaluasi menguji kinerja ketiga model menggunakan metrik *Accuracy*, *Precision*, *Recall*, dan *F1-Score*, lengkap dengan laporan klasifikasi dan penyimpanan *Confusion Matrix*. Berdasarkan tabel ringkasan metrik, *Gaussian Naive Bayes* memperoleh akurasi tertinggi sebesar 79% (F1-Score 0.6957), diikuti oleh *Decision Tree (max_depth=3)* sebesar 71%, dan *Decision Tree (max_depth=None)* terendah di angka 66% akibat kecenderungan *overfitting*.

4. Visualisasi:

Plot Pohon Keputusan dengan max_depth=3

Plot perbandingan metrik 3 model dalam bentuk bar chart

Jawaban :

Source Code dan Output :



tinggi namun tingkat kehadiran atau jam belajar yang rendah, sehingga batasan aturan model gagal menangkap kompleksitas kondisi aktual tersebut.

BAGIAN C : REKOMENDASI & ETIKA

Buat Laporan + Slide Presentasi 5 slide

1. Rekomendasi Model: Dari 3 model, mana yang kamu pilih untuk dipakai kampus? Jelaskan dengan data dari Bagian B

Jawaban :

Model yang Dipilih: Gaussian Naive Bayes

Penjelasan :

- Akurasi Tertinggi: Berdasarkan tabel evaluasi, *Gaussian Naive Bayes* mencatatkan Akurasi 79% dan Precision 83%, yang merupakan performa tertinggi dibandingkan *Decision Tree (max_depth=3)* sebesar 71% dan *Decision Tree (max_depth=None)* sebesar 66%.
- Menghindari Overfitting: Model *Decision Tree* dengan *max_depth=None* terbukti mengalami penurunan performa (overfitting) pada data uji. Oleh karena itu, *Gaussian Naive Bayes* lebih konsisten dan andal untuk diterapkan dalam memprediksi kelulusan mahasiswa secara objektif.

https://drive.google.com/file/d/14U_PSiNf2pnLtP5cCSUug_Z3FUKlTciD/view?usp=sharing

2. Bias & Etika: Fitur "Penghasilan_Ortu" dan "Jenis_Kelamin" rawan bias. Apa dampaknya jika model dipakai untuk beasiswa? Bagaimana mitigasinya?

Jawaban :

Dampak Jika Dipakai untuk Beasiswa: Fitur seperti "Penghasilan_Ortu" dan "Jenis_Kelamin" rawan memicu diskriminasi sistemik. Jika model digunakan untuk seleksi beasiswa, sistem berisiko mempelajari bias historis (misalnya, memperlakukan tidak adil kelompok gender tertentu atau mendiskriminasi mahasiswa dari tingkat penghasilan tertentu), sehingga keputusan beasiswa menjadi tidak objektif dan tidak adil.

Cara Mitigasinya:

- Melakukan Fairness Audit secara berkala untuk mengukur tingkat keadilan model terhadap kelompok demografis yang berbeda.
- Mempertimbangkan penghapusan fitur sensitif (*feature dropping*) jika fitur tersebut terbukti memicu bias tanpa relevansi langsung terhadap kapasitas

akademik murni.

- Menjadikan hasil prediksi model hanya sebagai alat bantu keputusan (*decision support tool*), bukan sebagai penentu mutlak tunggal dalam kelayakan beasiswa.

3. Simulasi: Jika 1 mahasiswa punya IPK tinggi tapi Kehadiran rendah, prediksi kedua model apa? Kenapa bisa beda.

Jawaban :

Source Code dan Output :

```
# =====
# 7. SIMULASI 1 DATA BARU (Bagian C.3)
# =====
print("\n" * 65)
print("7. SIMULASI: IPK TINGGI, KEHADIRAN RENDAH")
print("\n" * 65)

# Membuat 1 data baru: IPK tinggi (3.8), Kehadiran rendah (55%),
# nilai fitur lain diisi ke rata-rata/modus data training
sis_data = pd.DataFrame({
    "IPK": [3.80],
    "Kehadiran": [55.0],
    "Jan_Belajar": [X_train["Jan_Belajar"].median()],
    "Organisasi": [X_train["Organisasi"].mode()[0]],
    "Penghasilan_Ortu": [X_train["Penghasilan_Ortu"].mode()[0]],
    "Jenis_Kelamin": [X_train["Jenis_Kelamin"].mode()[0]],
    "Status_Beasiswa": [X_train["Status_Beasiswa"].mode()[0]],
})[X.columns]

for name in ["Decision Tree (max_depth=3)", "Gaussian Naive Bayes"]:
    model = trained_models[name]
    pred = model.predict(sis_data)[0]
    proba = model.predict_proba(sis_data)[0]
    label = target_le.inverse_transform([pred])[0]
    print(f"Nama Model: {name}")
    print(f"Prediksi : {label}")
    print(f"Probabilitas : "
          f"{' '.join(zip(target_le.classes_, np.round(proba, 3)))}")

print("\nSelesai. Semua hasil (gambar & csv) tersimpan di folder kerja.")
```

```
=====
7. SIMULASI: IPK TINGGI, KEHADIRAN RENDAH
=====

Decision Tree (max_depth=3)
Prediksi : Tidak Tepat Waktu
Probabilitas : {'Tepat Waktu': np.float64(0.0), 'Tidak Tepat Waktu': np.float64(1.0)}

Gaussian Naive Bayes
Prediksi : Tidak Tepat Waktu
Probabilitas : {'Tepat Waktu': np.float64(0.476), 'Tidak Tepat Waktu': np.float64(0.524)}

Selesai. Semua hasil (gambar & csv) tersimpan di folder kerja.
```

Penjelasan :

Tahap simulasi menguji satu data baru dengan kondisi kontradiktif, yaitu IPK tinggi (3.80) namun tingkat kehadiran rendah (55.0%). Berdasarkan hasil eksekusi program, baik model *Decision Tree* ($max_depth=3$) maupun *Gaussian Naive Bayes* sama-sama memberikan prediksi akhir "Tidak Tepat Waktu". Meskipun demikian, nilai probabilitas keduanya berbeda; *Decision Tree* sangat yakin dengan probabilitas 1.0 (100%) karena aturan percabangannya menganggap kehadiran rendah sebagai penalti mutlak, sedangkan *Gaussian Naive Bayes* menunjukkan probabilitas yang lebih seimbang dan ketat (47.6% Tepat Waktu vs 52.4% Tidak Tepat Waktu) karena memperhitungkan akumulasi probabilitas dari seluruh fitur secara bersamaan.

KESIMPULAN

Berdasarkan serangkaian proses implementasi, evaluasi, dan analisis data, dapat disimpulkan bahwa algoritma *Gaussian Naive Bayes* memberikan performa terbaik dalam memprediksi kelulusan mahasiswa tepat waktu dengan tingkat akurasi tertinggi sebesar 79% dan *precision* 83%. Meskipun demikian, model *Decision Tree* (khususnya dengan pembatasan $max_depth=3$) tetap unggul dalam hal transparansi karena aturan percabangannya mudah dibaca dan dijelaskan.

<https://drive.google.com/drive/folders/150u-kkC3DYch-Ow9uDQaB1ITgktKK5r?usp=sharing>

Github : https://github.com/Aggrakurnia/Triple_AI.git